Configurar Git localmente

vamos a generar un repositorio local para mi usuario de git necesito configurar usuario y correo por primera vez

Perfecto, vamos paso a paso como si fuera una guía de clase 👍

🔧 1. Verificar que tenés Git instalado

Primero asegurate que Git está funcionando:

```bash
git --version
```

Si devuelve una versión → OK
Si no, hay que instalar Git primero.

👤 2. Configurar usuario y correo (PRIMERA VEZ)

Git necesita identificar quién hace los commits.

👉 Configuración GLOBAL (recomendada)

Se aplica a todos tus repositorios:

```bash
git config --global user.name "Tu Nombre"
git config --global user.email "tuemail@ejemplo.com"
```

Ejemplo:

```bash
git config --global user.name "Mauricio Laurizi"
git config --global user.email "mauricio@email.com"
```

🔍 3. Verificar la configuración

```bash
```

```
git config --global --list
```

Deberías ver algo como:

```
user.name=Mauricio Laurizi
user.email=mauricio@email.com
```

📁 4. Crear un repositorio LOCAL

Elegí una carpeta o creala:

```bash
mkdir mi-proyecto
cd mi-proyecto
```

Inicializás el repo:

```bash
git init
```

Esto crea la carpeta oculta `.git` (ahí vive todo Git).

📌 5. Primer commit (opcional pero clave)

Creamos un archivo:

```bash
echo "# Mi Proyecto" > README.md
```

Lo agregamos:

```bash
git add README.md
```

Y hacemos commit:

```bash
git commit -m "Primer commit"
```

⚙️ 6. (Opcional) Configuración LOCAL por proyecto

Si querés que un proyecto tenga otro usuario (muy útil en docencia o trabajo):

```bash
git config user.name "Otro Nombre"
git config user.email "otro@email.com"
```

👉 Sin `--global` → solo afecta a ese repo

🧠 Concepto clave (para tus alumnos)

- `--global` → configuración del usuario en la máquina
- sin `--global` → configuración del repositorio
- `git init` → convierte una carpeta en repositorio
- `.git` → base de datos de versiones

Si querés en el próximo paso podemos:
✅ Conectarlo a GitHub
✅ Generar clave SSH
✅ O armarte una guía tipo TP para clase con esto 💡

mostrame la instruccion para pasar todos los archivos y prepararlos para un commit

Perfecto, esto es clave en Git 👇

📦 Agregar TODOS los archivos al staging (prepararlos para commit)

La instrucción es:

```bash
git add .
```

🔍 ¿Qué hace exactamente?

- Agrega **todos los archivos nuevos**

- Agrega **modificaciones de archivos existentes**
- Prepara todo en el **staging area** (área previa al commit)

👉 Es como decirle a Git:
"estos son los cambios que quiero guardar en el próximo commit"

⚠️ Alternativa más completa (recomendada para clase)

```bash
git add -A
```

✔️ Incluye:

- archivos nuevos
- archivos modificados
- archivos eliminados

👉 `git add .` a veces no toma eliminaciones dependiendo del contexto
👉 `git add -A` es más **consistente y profesional**

🔎 Verificar antes del commit

Siempre buena práctica:

```bash
git status
```

Te muestra:

- en verde → lo que ya está preparado
- en rojo → lo que todavía no agregaste

📌 Flujo completo típico

```bash
git add -A
git commit -m "Descripción de los cambios"
```

Si querés, en el siguiente paso te armo una explicación visual del **staging area vs working directory vs repo** (muy útil para tus clases 👍).

necesito generar mi primer commit y enviarlo al repo remoto

Perfecto, vamos a hacer el flujo completo: **primer commit + conexión al remoto + push** 🚀

📦 1. Preparar archivos (staging)

```bash
git add -A
```

📝 2. Crear el primer commit

```bash
git commit -m "Primer commit"
```

🔗 3. Conectar con el repositorio remoto

Primero necesitás crear un repo en GitHub (vacío, sin README si ya lo hiciste local).

Después vinculás tu repo local con el remoto:

```bash
git remote add origin https://github.com/usuario/nombre-repo.git
```

👉 Ejemplo:

```bash
git remote add origin https://github.com/mauricio/demo.git
```

🌿 4. Definir rama principal (si hace falta)

Hoy en día se usa `main`:

```bash
git branch -M main
```

🚀 5. Enviar al repositorio remoto

```bash
git push -u origin main
```

🔍 ¿Qué hace este comando?

- `push` → sube los cambios
- `origin` → nombre del remoto
- `main` → rama
- `-u` → deja configurado el seguimiento (después solo usás `git push`)

📌 Flujo completo resumido

```bash
git add -A
git commit -m "Primer commit"
git remote add origin https://github.com/usuario/repo.git
git branch -M main
git push -u origin main
```

⚠️ Posible problema (muy común)

Si GitHub te pide autenticación:

👉 Ya no se usa contraseña
👉 Tenés que usar:

- **Token (PAT)** o
- **SSH (más profesional)**

Si querés, en el siguiente paso te ayudo a:
✅ Configurar **SSH sin contraseña**
✅ O armarte esto como **TP para alumnos paso a paso**
✅ O resolver errores típicos de push (muy útil para clase)

pasame paso a paso para que los alumnos se clonen el repositorio en sus sistemas

Claro. Te dejo un **paso a paso simple para alumnos** para **clonar un repositorio remoto en su computadora**.

Guía para clonar un repositorio Git por primera vez

1. Verificar que Git esté instalado

Abrir la terminal o consola y escribir:

```bash
git --version
```

Si aparece un número de versión, Git ya está instalado.

Por ejemplo:

```bash
git version 2.49.0
```

---

2. Elegir la carpeta donde van a guardar el proyecto

Por ejemplo, pueden ir a **Documentos** o a una carpeta de trabajo creada para la materia.

Ejemplo en Windows:

```bash
cd Documents
```

O:

```bash
cd C:\Users\Alumno\Documents
```

También pueden crear una carpeta primero:

```bash
mkdir Programacion
cd Programacion
```

---

3. Copiar la URL del repositorio

Desde GitHub, GitLab o donde esté alojado el repositorio, buscar el botón **Code** y copiar la dirección.

Ejemplo:

```bash
https://github.com/usuario/proyecto-ejemplo.git
```

---

4. Clonar el repositorio

Usar el comando:

```bash
git clone URL_DEL_REPOSITORIO
```

Ejemplo:

```bash
git clone https://github.com/usuario/proyecto-ejemplo.git
```

Esto descargará una carpeta con todo el proyecto.

---

5. Entrar a la carpeta del proyecto

Después del clonado, entrar con:

```bash
```

```bash
cd proyecto-ejemplo
```

El nombre suele coincidir con el del repositorio.

---

6. Verificar que el repositorio se descargó correctamente

Pueden ejecutar:

```bash
git status
```

Si todo salió bien, Git mostrará algo como:

```bash
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Flujo completo resumido

```bash
git --version
cd Documents
git clone https://github.com/usuario/proyecto-ejemplo.git
cd proyecto-ejemplo
git status
```

Qué ocurre al clonar

Cuando el alumno ejecuta `git clone`:

- se descarga todo el proyecto
- se crea la carpeta del repositorio
- se descarga el historial de commits
- el repositorio remoto queda asociado automáticamente

Cómo actualizarlo después

Si el docente hace cambios en el repositorio, el alumno puede actualizarlos con:

```bash
git pull
```

Si además van a subir cambios

Antes de hacer commits por primera vez, cada alumno debería configurar su nombre y correo:

```bash
git config --global user.name "Nombre del Alumno"
git config --global user.email "correo@ejemplo.com"
```

Posibles errores comunes

Error 1: `git` no se reconoce

Significa que Git no está instalado o no está agregado al PATH.

Error 2: URL incorrecta

Verificar que la dirección copiada del repositorio sea correcta.

Error 3: repositorio privado

Si el repo es privado, el alumno necesitará permisos y autenticación.

Versión corta para entregar a alumnos

```bash
git clone https://github.com/usuario/proyecto-ejemplo.git
cd proyecto-ejemplo
git status
```

Texto explicativo para clase

Clonar un repositorio significa copiar en la computadora local un proyecto que está almacenado en un servidor remoto, incluyendo sus archivos y su historial de versiones.

Si querés, te lo puedo convertir en formato de **guía práctica para aula**, con apartados de **objetivo, pasos, comandos y actividad final**.

actualizame la información con el siguiente repositorio remoto para los alumnos: https://github.com/mauricioNico/universitas2026.git

Perfecto, te lo dejo ya **listo para entregar a los alumnos** con tu repositorio 👇

🧪 Guía práctica: Clonar repositorio de la materia

Repositorio a utilizar:
👉 GitHub
👉 https://github.com/mauricioNico/universitas2026.git

🎯 Objetivo

Clonar el repositorio de la materia en la computadora local para poder trabajar con los archivos del proyecto.

🔧 Paso 1: Verificar instalación de Git

Abrir la terminal (CMD, PowerShell o Git Bash) y ejecutar:

```bash
git --version
```

📁 Paso 2: Ubicarse en la carpeta de trabajo

Ejemplo:

```bash
cd Documents
```

O crear una carpeta para la materia:

```bash
mkdir Universitas
cd Universitas
```

⬇️ Paso 3: Clonar el repositorio

Ejecutar:

```bash
git clone https://github.com/mauricioNico/universitas2026.git
```

📂 Paso 4: Ingresar al proyecto

```bash
cd universitas2026
```

🔍 Paso 5: Verificar estado del repositorio

```bash
git status
```

Salida esperada:

```bash
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

🔄 Paso 6: Actualizar el proyecto (cuando haya cambios)

```bash
git pull
```

👤 Paso 7: Configurar usuario (solo la primera vez)

```bash
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"
```

📌 Flujo resumido

```bash
cd Documents
git clone https://github.com/mauricioNico/universitas2026.git
cd universitas2026
git status
```

🧠 Concepto clave

👉 **Clonar un repositorio** significa descargar:

- todos los archivos
- todo el historial de cambios
- la conexión con el repositorio remoto

⚠️ Problemas comunes

❌ "git no se reconoce"

👉 Git no está instalado

❌ Error de permisos

👉 El repositorio podría ser privado

❌ Carpeta ya existente

👉 Eliminarla o usar otro nombre

📝 Actividad sugerida

1. Clonar el repositorio
2. Crear un archivo llamado `mi_nombre.txt`
3. Escribir su nombre
4. Ejecutar:

```bash
git add .
git commit -m "Agrego mi archivo personal"
```

Si querés, en el siguiente paso te preparo:
✅ la guía para que ellos hagan **push correctamente (con token o SSH)**
✅ o un **TP completo evaluable con Git** (muy útil para clase)